Noc-Dashboard

Query & Data Reference

Issue	2
Date	2026-08-28
Prepared By	Noc-Dashboard Engineering

NOC-DASHBOARD

NOC-DASHBOARD ENGINEERING

Contents

Contents 2
1 Introduction 4
1.1 Two Data Sources 4
1.2 Query File Conventions 4
2 Local Cache Tables 5
2.1 Alarm Cache Tables 5
2.2 Site, POP & VIP Cache Tables 5
2.3 Partner Contact Cache Table 6
2.4 Incident & INC-WO Cache Tables 6
2.5 Telegram Alert Snapshot Tables 6
3 MSSQL — Alarm Feed 7
3.1 Reading the Alarm Feed 7
3.2 Acknowledging & Clearing Alarms 7
4 MSSQL — Site, POP & VIP Data 9
4.1 On-Air Site Master Data 9
4.2 POP Master Data 9
4.3 VIP / VVIP Site Master Data 9
5 MSSQL — Partner Contact 10
5.1 Reading Partners & Circuits 10
5.2 Creating, Editing & Deleting 10
6 PostgreSQL — Alarm Feed 12
6.1 ape_alarm 12
7 PostgreSQL — Incidents & INC-WOs 13
7.1 Incident Queries 13
7.2 INC-WO Queries 13
8 PostgreSQL — Site & Reference Data 15
8.1 Site Master Data 15
8.2 Technology, Vendor & Lookup Tables 15
9 Cambodia Geo Data 17

9.1 The object_N Naming Scheme....17
9.2 How the Hierarchy Is Used....17
9.3 External Boundary Data....17
10 Map View Color Reference....19
10.1 Site Dot Colors....19
10.2 Location Search Section Colors....19

1 Introduction

This chapter explains where Noc-Dashboard's data comes from and how its query files are organized.

1.1 Two Data Sources

Noc-Dashboard reads from two independent upstream databases, never directly from a page — every query result is synced into a local SQLite cache on a background schedule, and pages only ever read that cache.

Source	Used for	Access pattern
MSSQL (noc schema)	ISP POP / EZE / TCT / IT NODE alarms, POP master data, On-Air site master data, VIP/VVIP sites, Partner Contact	pymssql, named queries in app/database/queries/*.sql
PostgreSQL (Streamline)	Incidents, INC-WOs, the broader alarm feed, site master data, technology/vendor inventory	SQLAlchemy against generically-named object_N foreign tables

NOTE

PostgreSQL's tables are named object_1, object_2 ... object_438 with no meaning of their own — see Chapter 9 for how a real table name is looked up and the specific tables this app depends on.

1.2 Query File Conventions

Each .sql file under app/database/queries/ holds one or more named queries, split by a -- name: marker comment. app/database/sql_loader.py reads a file once and returns the query text matching a given name, so multiple related queries (e.g. every Partner Contact CRUD statement) can share one file.

Almost every query is read-only. The only two exceptions — acknowledge_note and clear_alarm in alarms.sql — write back to MSSQL directly, since acknowledging or clearing an alarm has to reach the real source system, not just the local cache.

2 Local Cache Tables

instance/localstorage.db is a single SQLite file holding every table below. Each is owned by one background scheduler, rebuilt in full (replace_all, not incremental) on its own interval — pages, drilldowns, and CSV/XLSX exports only ever read from here, never from MSSQL or PostgreSQL directly.

2.1 Alarm Cache Tables

Cache Table	Source	Refreshed By
isp_pop_alarm_cache	MSSQL noc.TBL_PPD + TBL_ISPDEV + TBL_POP	alarm_scheduler
eze_alarm_cache	MSSQL noc.TBL_PPD + TBL_ISPDEV + TBL_POP	alarm_scheduler
tct_alarm_cache	MSSQL noc.TBL_PPD + TBL_ISPDEV + TBL_POP	alarm_scheduler
it_node_alarm_cache	MSSQL noc.TBL_PPD	alarm_scheduler
ape_alarm	PostgreSQL object_138	ape_alarm_scheduler
vip_alarm	Filtered from ape_alarm in Python (VIP/VVIP sites only) — not its own query	vip_alarm_scheduler

NOTE

vip_alarm and ape_alarm used to run two independent full scans of object_138 on separate schedules. vip_alarm is now built by filtering the already-fetched ape_alarm rows instead, avoiding a second scan of the same 10M-row table.

2.2 Site, POP & VIP Cache Tables

Cache Table	Source	Refreshed By
site_cache	PostgreSQL object_160 + object_162/164/166/168/308	site_scheduler
mssql_site_onair	MSSQL TBL_SITE + TBL_NE	mssql_site_onair_scheduler
pop_cache	MSSQL noc.TBL_POP	pop_scheduler
vip_site_cache	MSSQL SITE_INFO	vip_site_scheduler
site_lookup_cache	PostgreSQL site_lookup_mv (materialized view)	site_scheduler
site_technology_cache	PostgreSQL object_227 + object_160	site_scheduler
ne_nms_lookup_cache	PostgreSQL object_227 + object_160	site_scheduler
site_ran_vendor_cache	PostgreSQL object_160	site_scheduler
counterparty_cache	PostgreSQL object_170	site_scheduler

2.3 Partner Contact Cache Table

Cache Table	Source	Refreshed By
partner_contact_cache	MSSQL TBL_PARTNER + TBL_PARTNER_CONTACT + TBL_PARTNER_CIRCUIT	partner_contact_scheduler

2.4 Incident & INC-WO Cache Tables

Cache Table	Source	Refreshed By
incident_by_status	PostgreSQL object_322 + 160/162/164/166/308	incident_scheduler
incident_site_down	PostgreSQL object_322 + 160/162/164/166/308	incident_scheduler
vip_incident_status	PostgreSQL object_322 + 160/162/164/166/308, VIP/VVIP scoped	incident_scheduler
incident_created_trend	PostgreSQL object_322	incident_scheduler
incident_closed_trend	PostgreSQL object_322	incident_scheduler
inc_wo_by_status	PostgreSQL object_324 + 322/310/160/162/164/166/308	incident_scheduler
inc_wo_site_down	PostgreSQL object_324 + 322/310/160/162/164/166/308	incident_scheduler
inc_wo_category_power	PostgreSQL object_324 + 322/310/160/162/164/166/308	incident_scheduler
inc_wo_resolved	PostgreSQL object_324 + object_322	incident_scheduler

2.5 Telegram Alert Snapshot Tables

These four tables don't come from their own database query — each one mirrors exactly what its Telegram job last sent, at the moment it sent it, so the last alert is always inspectable independently of the dashboard's own cards.

Cache Table	Mirrors	Refreshed By
telegram_alert_cache	incident_site_down, at send time	telegram_alert.py's own schedule
telegram_inc_wo_alert_cache	inc_wo_site_down, at send time	telegram_inc_wo_alert.py's own schedule
telegram_inc_wo_power_alert_cache	inc_wo_category_power, at send time	telegram_inc_wo_power_alert.py's own schedule
telegram_inc_wo_general_alert_cache	inc_wo_by_status, other categories, at send time	telegram_inc_wo_general_alert.py's own schedule

3 MSSQL — Alarm Feed

The ISP POP, EZE, TCT, and IT NODE dashboard cards all read from one MSSQL table, noc.TBL_PPD, filtered differently per alarm family. Defined in alarms.sql.

3.1 Reading the Alarm Feed

Query	Source Table(s)	Feeds
isp_pop_alarm_cache	noc.TBL_PPD + TBL_ISPDEV + TBL_POP	ISP POP card — AlarmName IN ('POP/PON DOWN', 'POP/PON POWER'), still open
eze_alarm_cache	noc.TBL_PPD + TBL_ISPDEV + TBL_POP	IP Ezecom card — AlarmName LIKE 'EZE%', still open
tct_alarm_cache	noc.TBL_PPD + TBL_ISPDEV + TBL_POP	IP Telcotech/Cellcard card — AlarmName LIKE 'TCT%', still open
it_node_alarm_cache	noc.TBL_PPD	IT NODE card — AlarmName = 'IT_NODE', still open

The first three all join noc.TBL_PPD to noc.TBL_ISPDEV and noc.TBL_POP to resolve a POP name/code/province onto each alarm row. IT NODE alarms are IT-infrastructure monitoring, unrelated to any POP site, so it_node_alarm_cache skips that join entirely.

3.2 Acknowledging & Clearing Alarms

acknowledge_note and clear_alarm are the only two write-back queries in the whole application — every other query anywhere in this app is read-only. Both write directly to noc.TBL_PPD.

Query	What it does
acknowledge_note	Writes the Acknowledge/Modify popup's note back to the row. First-ever acknowledgment stamps ACK/AB; every later edit stamps MD/MB instead — decided per-row from ACK's own pre-update value, so two staff submitting for the same row at once still resolves to one consistent outcome.
clear_alarm	Manually marks a row cleared (Status = 0, RestoredTime = now), reproducing what happens when the source device clears the alarm itself. Only affects a row still genuinely open (RestoredTime IS NULL) — if the device already cleared it first, nothing is overwritten.

NOTE

Both are matched by ID, the row's real uniqueidentifier primary key — not RTID, which was confirmed live to repeat across unrelated rows and once caused an Ack to silently update 4 unrelated alarms instead of the intended 1.

4 MSSQL — Site, POP & VIP Data

Master data for On-Air sites, POPs, and VIP/VVIP sites, each its own local cache table refreshed on its own schedule.

4.1 On-Air Site Master Data

Query	Source Table(s)	Cache Table
site_onair	TBL_SITE + TBL_NE	mssql_site_onair

Joins TBL_SITE to TBL_NE for VENDOR, scoped to STATUS = 'On Air'. This is a separate source from PostgreSQL's own site_cache — MSSQL has no 'planned' or 'ready_for_installation' concept and no hub_site field at all, which is why the Map plots On-Air sites from here and Planned/Other sites from PostgreSQL independently rather than merging the two (see Chapter 10).

4.2 POP Master Data

Query	Source Table(s)	Cache Table
pop_cache	noc.TBL_POP	pop_cache

Reads noc.TBL_POP in full — the same table alarms.sql already joins against for pop_name/pop_code/province, mirrored here for its own local cache of every non-removed POP.

4.3 VIP / VVIP Site Master Data

Query	Source Table(s)	Cache Table
vip_site_cache	SITE_INFO	vip_site_cache

Reads SITE_INFO where VIP IN ('VIP', 'VVIP') — the master list behind the Dashboard's VVIP Site card.

5 MSSQL — Partner Contact

Partner Contact is backed entirely by MSSQL (TBL_PARTNER / TBL_PARTNER_CONTACT / TBL_PARTNER_CIRCUIT), defined in partner-contact.sql.

5.1 Reading Partners & Circuits

Query	Source Table(s)	What it returns
partner_contact_cache	TBL_PARTNER + TBL_PARTNER_CONTACT + TBL_PARTNER_CIRCUIT	Every partner joined to its contacts, with each partner's circuits pre-aggregated into one comma-separated Circuit IDs / Peering Infos column — the query behind the cached Partner Contact list.
fetch_partner_by_id	TBL_PARTNER	One partner's header info (name, type) for the Manage Circuits page — queried live, not from the cache.
fetch_circuits_for_partner	TBL_PARTNER_CIRCUIT	Itemized circuit rows for one partner, queried live since the cache only stores circuits pre-aggregated into text and has no per-circuit id to edit or delete against.

5.2 Creating, Editing & Deleting

Eight write queries back the Partner Contact page's forms — creating a partner, adding a contact, editing either, deleting a contact, and the matching four for circuits.

Query	Table	Effect
insert_partner	TBL_PARTNER	Creates a new Partner row. Its PartnerID is generated in Python, not by SQL, so the same id can be reused across every insert_contact call that follows it in the same transaction.
insert_contact	TBL_PARTNER_CONTACT	Adds one Contact to an existing Partner — never touches the Partner row itself, so a partner can end up with several escalation-level contacts sharing one PartnerID.
update_partner / update_contact	TBL_PARTNER / TBL_PARTNER_CONTACT	Standard field updates by PartnerID / ContactID.

delete_contact	TBL_PARTNER_CONTACT	Removes just that Contact row — the Partner and its other contacts are untouched.
insert_circuit / update_circuit / delete_circuit	TBL_PARTNER_CIRCUIT	Same create/edit/delete shape as contacts, scoped to circuits.

6 PostgreSQL — Alarm Feed

The broader alarm feed behind several RAN/Alarm cards, read from object_138 (the [APE] Alarm table).

6.1 ape_alarm

Query	Source Table(s)	Cache Table
ape_alarm	object_138	ape_alarm

Selects every currently-open row from object_138, with no date window and no event-name filter, then deduplicates in SQL: a ROW_NUMBER() window keeps only the latest occurrence per (site, event_name, location), while a matching COUNT(*) records how many times that combination actually fired as an Occurrence Count column.

NOTE

Two narrower alternatives were tried and reverted — skipping dedup for 'cell' events, and a category-based partial dedup — both let the same 'thousands of near-identical rows' problem back in, so full dedup on every row is the deliberate default for the 9+ services that share this cache.

Site Name is resolved afterward in Python against site_cache, not in this query — see app/services/ape_alarm/ape_alarm_sync_service.py.

NOTE

Each ape_alarm sync retries automatically on failure, with capped exponential backoff (2s, 4s, 8s, 16s, 30s, capped) for up to 8 attempts — worst case roughly two minutes — before giving up for that cycle. This mitigates 'conflict with recovery', where a long-running scan against object_138 loses its race with the replica's WAL replay; a cycle that exhausts every attempt leaves the existing cache untouched and simply retries on the next scheduled cycle.

7 PostgreSQL — Incidents & INC-WOs

Incident and INC-WO queries share a consistent shape: a join from object_322 (Incident) or object_324 (INC-WO) out to Site/Province/District/Commune, each scoped to the specific status or category one dashboard card needs.

7.1 Incident Queries

Query	Source Table(s)	Scope	Feeds
incident_by_status	object_322 + 160/162/164/166/308	status IN (new, in_progress)	Every Incident-column badge on the Dashboard
incident_site_down	object_322 + 160/162/164/166/308	category = site_down, not closed, alarm not yet cleared	Incident Site Down card and the Telegram Site Down alert
vip_incident_status	object_322 + 160/162/164/166/308	VIP/VVIP sites only, not closed (Resolved included)	VIP Site incident badges
incident_created_trend (+ _by_ids)	object_322	Created in the last 15 days, any status	Incident Volume — Week over Week chart
incident_closed_trend (+ _by_ids)	object_322	Closed in the last 15 days	Incidents Closed — Week over Week chart

NOTE

> incident_closed_trend reads object_322's own closed_at directly rather than the work order's resolved_at — confirmed live that roughly a third of one sample day's closures had no work order at all, which made the earlier work-order-driven version silently miss them rather than just delay them.

The two trend queries are deliberately split into two steps: a plain single-table SELECT against object_322 alone first, then a second _by_ids query that joins in Site/Province/District/Commune detail only for the specific batch of ids returned. Joining two foreign tables directly in one query was confirmed live to fail under replica-recovery load on every attempt (0 of 18 across 3 rounds), while a plain filtered SELECT against one foreign table alone consistently succeeds.

7.2 INC-WO Queries

Query	Source Table(s)	Scope	Feeds
inc_wo_by_status	object_324 + 322/310/160/162/164/166/308	status != closed, parent	INC-WO by Status card

		Incident not closed	
inc_wo_site_down	object_324 + 322/310/160/162/164/166/308	category = site_down, not resolved/closed	INC-WO Site Down card, graded by Hub/Diamond/Gold/Silver/Bronze
inc_wo_category_power	object_324 + 322/310/160/162/164/166/308	category = power, not resolved/closed	Telegram power alert only — no dashboard card
inc_wo_resolved	object_324 + object_322/310/160/162/164/166/308	status = resolved, parent Incident still in_progress	INC-WO Resolved card
inc_wo_by_incident_ids	object_324 + object_310	Scoped to a batch of incident ids, no status filter	Resolves 'latest INC-WO per incident/site' for the Incident drilldowns

8 PostgreSQL — Site & Reference Data

Site master data, technology inventory, and vendor reference tables — the data every Incident/INC-WO query above joins out to.

8.1 Site Master Data

Query	Source Table(s)	Cache Table
site_cache	object_160 + 162/164/166/168/308	site_cache

site_cache is the curated site master record behind the Map's Planned/Other markers — name, status, grade, coordinates, hub_site, and actual_ran_vendor, joined out to Province/District/Commune/Village and Physical Location. Only sites with status != off_air are kept.

8.2 Technology, Vendor & Lookup Tables

Query	Source Table(s)	Purpose
ne_nms_lookup	object_227 + object_160	One Site Code -> NMS vendor code per site, resolved once here instead of recomputing the same window function inside 6 other query files.
site_technology_cache	object_227 + object_160	Every distinct (site, technology) pair — a site's full 2G/3G/4G/5G footprint, needed to tell a completely-down site apart from one still up on another technology.
site_ran_vendor	object_160	actual_ran_vendor — a site-level 'who runs this site's RAN today' field, kept separate from the NE-level inventory above since equipment/vendor swaps can leave that inventory stale.
site_lookup_cache	site_lookup_mv (materialized view)	sran_id/nodeb_id/bts_id -> site_name, synced so app/utils/get_processed_site.py can resolve raw NE identifiers in Python instead of calling Postgres's own per-row function on every alarm.
counterparty	object_170	The platform's general vendor/brand catalog — cached for reference only; not treated as authoritative for

		RAN vendor names (see that query's own header for a known Huawei/Nokia mismatch).

9 Cambodia Geo Data

Province, District, Commune, and Village are the geographic hierarchy every site, incident, and alarm ultimately resolves against.

9.1 The object_N Naming Scheme

PostgreSQL stores its data in generically-named foreign tables — object_1 through object_438 — with no meaning of their own. object_1 is the platform's own model registry; every other object_N table has a matching row in it, keyed by id, so a table's real name can always be looked up directly.

Table	Real name	Alias
object_160	Site	site
object_162	Province	province
object_164	District	district
object_166	Commune	commune
object_168	Village	village
object_308	Physical Location	ploc

9.2 How the Hierarchy Is Used

Every site-level query in this app resolves the same four-level chain — Province, District, Commune, Village — by joining a site's (or an incident's own, via its Other-Site fallback) province/district/commune/village foreign keys out to their name tables. This is what lets the Map's boundary search work: searching a province, district, or commune scopes the site list to a matching name at that level, drawn straight from these same tables.

NOTE

A site that isn't registered in object_160 falls back to the parent Incident's own province_ref/district/commune fields instead — a CASE on whether the site join matched, not a COALESCE on the field itself, since some registered sites also have a genuinely blank location of their own.

9.3 External Boundary Data

object_162/164/166/168 give names only, no polygon geometry — Province/District/Commune/Village are name lookups, not shapes. Real Cambodia commune boundary polygons aren't stored anywhere in this app; the table below is an external, openly-licensed source for anyone who needs them (e.g. drawing real commune outlines on the Map instead of just point markers).

Dataset	Format	License	Link
Cambodia Commune	GeoJSON	CC BY 4.0	Download

Boundaries (ADM3) — geoBoundaries			
Cambodia Commune Boundaries (ADM3), all formats — geoBoundaries	ZIP (GeoJSON, TopoJSON, Shapefile)	CC BY 4.0	Download

NOTE

Not part of the application or its database — an external reference only, from the geoBoundaries open political-boundaries project (geoboundaries.org), 1,633 commune-level units for Cambodia. Verify against the live geoBoundaries API (/api/current/gbOpen/KHM/ADM3/) before relying on it for anything beyond casual reference, since upstream releases can update independently of this document.

10 Map View Color Reference

Every site is plotted individually with no clustering, colored by status, with an open Site Down incident always overriding the color below it.

10.1 Site Dot Colors

Based on site_cache.status, with is_down (an open Site Down incident) overriding all of them. Defined in map-view.js's DOT_COLORS/DEFAULT_DOT_COLOR/DOWN_COLOR, mirrored in map-view.css's .map-dot-* classes.

Color	Hex	Meaning
Red	#ef4444	is_down — an open Site Down incident (overrides every status color below)
Green	#10b981	status = on_air
Blue	#0487e2	status = planned
Grey	#94a3b8	anything else — Other

10.2 Location Search Section Colors

The Map's location search dropdown groups results by Province / District / Commune / Village. Color is applied to each group's section title only, not per result row.

Color	Hex	Level
Red	#ef4444	Province
Violet	#8b5cf6	District
Blue	#2563eb	Commune
Amber	#f59e0b	Village